

Article

Energy Demand Response in a Food-Processing Plant: A Deep Reinforcement Learning Approach

Philipp Wohlgenannt ^{1,2,*} , Sebastian Hegenbart ^{3,4} , Elias Eder ¹ , Mohan Kolhe ²  and Peter Kepplinger ¹ 

¹ Josef Ressel Centre for Intelligent Thermal Energy Systems, Illwerke vkw Endowed Professorship for Energy Efficiency, Energy Research Centre, Vorarlberg University of Applied Sciences, Hochschulstrasse 1, 6850 Dornbirn, Austria; elias.eder@fhv.at (E.E.); peter.kepplinger@fhv.at (P.K.)

² Faculty of Engineering and Science, University of Agder, Jon Lilletuns vei 9, 4879 Grimstad, Norway; mohan.l.kolhe@uia.no

³ Department of Engineering and Technology, Vorarlberg University of Applied Sciences, Hochschulstrasse 1, 6850 Dornbirn, Austria; sebastian.hegenbart@fhv.at

⁴ Digital Factory Vorarlberg GmbH, Hochschulstrasse 1, 6850 Dornbirn, Austria

* Correspondence: philippw@uia.no

Abstract: The food industry faces significant challenges in managing operational costs due to its high energy intensity and rising energy prices. Industrial food-processing facilities, with substantial thermal capacities and large demands for cooling and heating, offer promising opportunities for demand response (DR) strategies. This study explores the application of deep reinforcement learning (RL) as an innovative, data-driven approach for DR in the food industry. By leveraging the adaptive, self-learning capabilities of RL, energy costs in the investigated plant are effectively decreased. The RL algorithm was compared with the well-established optimization method Mixed Integer Linear Programming (MILP), and both were benchmarked against a reference scenario without DR. The two optimization strategies demonstrate cost savings of 17.57% and 18.65% for RL and MILP, respectively. Although RL is slightly less efficient in cost reduction, it significantly outperforms in computational speed, being approximately 20 times faster. During operation, RL only needs 2 ms per optimization compared to 19 s for MILP, making it a promising optimization tool for edge computing. Moreover, while MILP's computation time increases considerably with the number of binary variables, RL efficiently learns dynamic system behavior and scales to more complex systems without significant performance degradation. These results highlight that deep RL, when applied to DR, offers substantial cost savings and computational efficiency, with broad applicability to energy management in various applications.

Keywords: industrial energy-management systems; demand response; reinforcement learning; machine learning; double deep Q-learning



Citation: Wohlgenannt, P.; Hegenbart, S.; Eder, E.; Kolhe, M.; Kepplinger, P. Energy Demand Response in a Food-Processing Plant: A Deep Reinforcement Learning Approach. *Energies* **2024**, *17*, 6430. <https://doi.org/10.3390/en17246430>

Academic Editor: Frede Blaabjerg

Received: 22 November 2024

Revised: 16 December 2024

Accepted: 18 December 2024

Published: 20 December 2024



Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Industrial food-processing facilities are energy intensive, where up to 50% of food-production costs are energy-related [1]. One effective approach to reduce energy costs, lower emissions, or enhance energy efficiency is DR for load profile adjustments [2]. The enormous DR potential of the industrial sector is shown in Siddiquee et al. [3]. In the food industry, this DR potential exists due to the high energy demand for heating and cooling food products [4]. One promising approach to use this load-shifting potential is via thermal energy storages (TESs) [5]. In this study, a food-processing plant with a TES is optimized using RL. The first part of the introduction focuses on DR in food-processing plants using thermal capacities as TES followed by RL as a general optimization method and some DR applications of RL in various fields. Finally, the research gap is highlighted and the contributions are shown.

1.1. Food-Processing Plants for Demand Response

This subsection summarizes the literature of DR applications in food-processing plants. Chen et al. [6] developed an energy hub model to simulate the electricity demand of a food-processing procedure within an industrial park. They implemented a two-stage robust co-optimization approach using MILP to minimize electricity costs under a time-of-use (TOU) pricing scheme. This case study used a battery energy-storage system and a TES for flexibility. Giordano et al. [7] examined the energy supply of a milk-production plant. They modeled the energy supply using fixed load profiles and incorporated energy storage for flexibility. A rule-based operation optimized energy consumption, reducing energy demand and carbon emissions. Pazmiño-Arias et al. [8] optimized the energy supply of a dairy factory using an energy hub model with fixed load profiles for heat, cooling, and electricity. They applied nonlinear optimization to minimize total energy costs in a TOU pricing scenario. Cirocco et al. [9] investigated the DR potential of combining industrial TES with a photovoltaic (PV) system in an Australian winery. Using historical load profiles, the authors utilized a TES to boost PV self-consumption. A nonlinear optimal control algorithm reduced electricity costs by 22% compared to a non-optimized case without PV. Saffari et al. [10] conducted a case study in a dairy factory that produces yogurt and cheese. The work investigated the combination of TES and PV using an MILP formulation based on load profiles. The simulations showed cost reductions from 1.5% to 10%, depending on the scenario. In an earlier study conducted by the authors of this paper [11,12], an Austrian food-processing plant was modeled and its operation was optimized in a simulation study. The study used a detailed model of the food-production process for DR. Flexibility options such as thermal mass or a chilled water buffer were compared. The DR optimization problem was solved via MILP, achieving reductions in electrical power consumption by up to 18%, electricity costs by up to 24%, and peak load by up to 36%. In summary, the literature shows a clear potential for DR in food-processing plants.

1.2. Reinforcement Learning for Demand Response

Literature reviews such as Zhang and Grossmann [13] from 2016 and the literature from Section 1.1 show that MILP is the standard method for optimization in industrial DR problems. In Zhang and Grossmann [13], 36 of 42 investigated articles on industrial demand side management (DSM) used MILP. Although MILP is computationally efficient for small problems and is capable of finding the global optimum, a large number of binary variables can drastically increase the computation time. Furthermore, a downside of MILP is that the problem has to be solved separately for every different time window in model predictive control (MPC). In contrast, machine learning (ML) techniques such as RL require training only once, resulting in a computationally efficient application afterwards. Therefore, the potential of RL in a food-processing plant is investigated in this study.

Recently, promising technologies like Deep Q-Learning (DQL) have emerged. DQL, introduced by Mnih et al. [14], replaces the Q-table from classical Q-learning [15] with a neural network, enabling it to handle high-dimensional state spaces. DQL showed enormous potential, being capable of achieving great scores in over 2600 Atari games [14]. A limitation of the DQL network is its tendency to overestimate state values and moving targets. To solve this problem, van Hasselt et al. [16] introduced double deep Q-learning (DDQL). They adapted the DQL network by using one neural network for choosing the best action and one for estimating the value the next action. Thereby, they achieved better or similar scores in the Atari games compared to the DQL. RL's application scope has rapidly expanded, making it a key research focus in various fields. Vazquez et al. [17] investigated the application of traditional RL algorithms such as Q-learning or Monte Carlo for DR and concluded that these RL algorithms are capable in DR to help integrate renewable energies into the power grid. Deep reinforcement learning (DRL) such as DQL and DDQL for smart building energy management are investigated in a review paper [18], showing that nearly all model-free DRL-based building energy-optimization methods are still not implemented in practice. Furthermore, DRL can be used in buildings to simultaneously

reduce energy cost, peak load, and occupant dissatisfaction degree. A notable example of RL in practice is Google DeepMind's achievement of a 40% reduction in data center cooling costs using RL techniques [19]. In the following part of the literature review, the classification from OpenAI [20] is used where model-free RL is categorized into policy-based a.k.a. policy optimization, value-based a.k.a. Q-Learning, or methods based on both. There, DQL and DDQL are listed as deep Q-networks (DQN). In DR literature, value-based RL has been applied in residential [21–24] or commercial buildings [25–27], smart grids [28], or in microgrids [29]. The applied RL algorithms are based on QL [21,28,30] or DQN [22–27,29] and used to improve the energy efficiency or to reduce the energy costs. Value- and policy-based optimization was applied in commercial buildings [31–33], in a laboratory setup [34], or in an industrial warehouse [35]. The applied RL algorithms are based on soft actor-critic (SAC) [31–33], deep deterministic policy gradient (DDPG) [34], or random augmented search [35]. SAC was successfully implemented in a real office building in showing a decrease of temperature violation by 68% while maintaining a similar energy consumption as the reference case [33]. Policy-based RL with proximal policy optimization (PPO) was applied in a commercial building [36] or a university [37]. Policy-based RL algorithms are well-suited for continuous action spaces, but value-based RL like DQN is preferred for discrete action spaces for its lower computation costs and greater sample efficiency. Therefore, DQN has potential for application in energy management, because heating, ventilation, and air conditioning (HVAC) systems often use digital controllers with a discrete set point.

1.3. Contributions

In summary, RL shows great potential in various application fields. DDQL has not yet been applied to optimize the set point temperature of a refrigerated warehouse in a food-processing plant for DR using real-time pricing (RTP) and compared with a state-of-the-art MPC controller based on MILP. We investigated an on-site warehouse in a food-processing plant. This warehouse is influenced not only by ambient temperature and the HVAC system, but also by the food-production process, which significantly impacts the system's dynamics. We used the thermal capacity of the ware and the building as TES and applied DR by optimizing the set point temperature of the warehouse's proportional integral (PI) controller. Our main contributions are:

- Instead of directly controlling the cooling power, the set point temperature of a PI controller was optimized. This enhances stability and simplifies practical implementation.
- DDQL—a state-of-the-art RL algorithm—for load shifting was applied to reduce energy costs in an RTP scenario.
- The problem was additionally formulated as MILP to compare RL with a state-of-the-art MPC controller.
- The respective energy cost savings and computation times of RL and MILP were evaluated.

The remainder of this study is organized as follows. The system model for RL and MILP, the simulation study concept, the RL algorithm, and the MILP are described in Section 2. In Section 3, the optimization results are presented, including an analysis of the RL training process and the influence of the price signal on the DR potential. Finally, conclusions are drawn in Section 4.

2. Methods

In this study, we investigated a food-processing plant, where cheese is produced and stored. The focus of the paper is the industrial warehouse in the plant, which is used to cool and store cheese. The products of multiple months are stored in a high-bay racking. Cheese enters the warehouse at temperatures between 80 °C and 130 °C and is cooled to approximately 0 °C for storage until sold. An industrial refrigeration system handles this cooling process and the thermal mass of the warehouse, including the stored cheese products, functions as a TES. By allowing a defined temperature range between 0 °C and

5 °C, this TES provides flexibility for optimization. The goal of the optimizer is to shift the electrical load of the refrigeration plant to periods with lower electricity prices, reducing overall energy costs. Instead of directly controlling the cooling power, the optimizer adjusts the set point temperature of an existing PI controller. We compared two optimization approaches: dynamic optimization using RL and classical MILP. The optimization is incentivized using hourly RTP. For comparison, we used a realistic, currently employed scenario where the set point temperature remains constant at 2.5 °C as a reference case.

2.1. Data Acquisition and Processing

We used hourly data between May 2020 and April 2023 from an industrial plant, including production data (the mass flow of product in the warehouse due to production $\dot{m}_{in}$), ambient temperature (T_{∞}), the energy efficiency ratio of the refrigeration system (β), and approximations of heat losses to the ambient ($\dot{Q}_{loss}$). The hourly spot market prices from Energy Exchange Austria (EXAA) [38] was used as RTP for the optimization process. The electricity price and the predicted heating load are assumed to be known for the current day. The data were split into two sets: two years of data were used to train the RL agent, while one year was reserved for testing, either via MILP or by the RL agent. The simulation study was conducted in Python.

2.2. Industrial Warehouse Model

Figure 1 shows an overview of the system. The industrial warehouse was modeled with the thermal capacity C_H of the complete warehouse and a transient energy balance. The thermal capacity is the sum of the internal thermal capacities of the stored cheese products and the thermal mass of the building, where the steel of the high-bay racking is a relevant factor. This is modeled by

$$C_H(t) = c_{cheese}m_{cheese}(t) + c_{building}m_{building} \quad (1)$$

where E is the energy, c_{cheese} and $c_{building}$ are the specific thermal capacities of cheese and the building, and m_{cheese} and $m_{building}$ are the masses of the cheese and the building, respectively. The temperature of the industrial warehouse T_H can be calculated via a transient energy balance:

$$\frac{dE}{dt} = C_H(t)\dot{T}_H(t) = \dot{Q}_{gain}(t) + \dot{m}_{in}(t)h(T_{in}) - \dot{m}_{out}(t)h(T_{out}) - \dot{Q}_{cool}(t) \quad (2)$$

where $\dot{Q}_{gain}(t)$ is the heat gain from ambient, $\dot{m}_{in}(t)$ is the mass flow rate of hot cheese in the warehouse, $\dot{m}_{out}(t)$ is the mass flow rate of cold cheese from the warehouse, $h(T)$ is the enthalpy of the cheese, and $\dot{Q}_{cool}(t)$ is the cooling power of the chiller. The chiller is modeled via an average energy efficiency rate β :

$$\dot{Q}_{cool}(t) = P_{el}(t)\beta \quad (3)$$

Assuming that $\dot{m}_{in}(t)$ and $\dot{m}_{out}(t)$ are equal, C_H is constant. As a further simplification, T_{in} and T_{out} of the cheese are assumed to be constant. Thereby, the heat flow rates can be simplified to:

$$\dot{Q}_{heat}(t) = \dot{Q}_{gain}(t) + \dot{m}_{in}(t)h_{in} - \dot{m}_{out}(t)h_{out} \quad (4)$$

Applying all simplifications mentioned, this leads to:

$$\dot{T}_H(t) = \frac{1}{C_H}(\dot{Q}_{heat}(t) - P_{el}(t)\beta) \quad (5)$$

Applying discretization to the inputs ($\dot{Q}_{\text{heat}}(t), P_{\text{el}}(t)$) for a specified time interval i of duration Δt , the transient thermal energy balance can be simplified to:

$$T_{H,i+1} = T_{H,i} + \frac{1}{C_H} (\dot{Q}_{\text{heat},i} - P_{\text{el},i} \beta) \Delta t \quad (6)$$

Equation (6) can be used to calculate the system behavior.

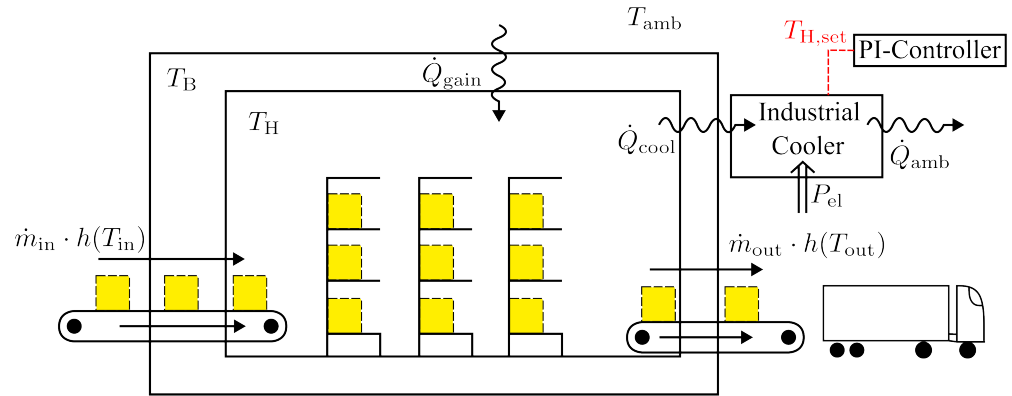


Figure 1. Scheme of the food-processing plant including the building envelope and the warehouse showing considered mass flows, heat flows, and the electrical power of the industrial cooler.

The cooling power $\dot{Q}_{\text{cool}}(t)$ is controlled through a basic time-discrete PI controller with saturation, as shown in Algorithm 1:

Algorithm 1 PI controller with saturation

```

1:  $u_i = k_P(T_{H,i} - T_{\text{set},i}) + (k_I \Delta t - k_P)(T_{H,i-1} - T_{\text{set},i-1}) + u_{j-1}$ 
2: if  $P_{\text{el},i} > P_{\text{max}}$  then
3:    $P_{\text{el},i} = P_{\text{max}}$ 
4: else if  $P_{\text{el},i} < P_{\text{min}}$  then
5:    $P_{\text{el},i} = P_{\text{min}}$ 
6: else
7:    $P_{\text{el},i} = u_i$ 
8: end if

```

where $T_{\text{set},i}$ is the temperature of the set point, k_P is the proportional factor, k_I is the integral factor, P_{max} is the maximum power, and P_{min} is the minimum power, u_i is the output signal of the PI controller before saturation, and $P_{\text{el},i}$ is the output signal after saturation that is used to control the electrical power of the chiller. In the reference case, $T_{\text{set},i}$ is constant. In the scenarios RL and MILP, the $T_{\text{set},i}$ can be between a lower boundary T_{lb} and an upper boundary T_{ub} . The resulting flexibility in temperature associated with the thermal capacity C_H is used for load shifting. The model parameters are shown in Table 1.

The specific thermal capacity c_{cheese} was taken from ref. [12], the energy efficiency rate β is a yearly average based on historical data, the temperature band is defined from 0 °C to 5 °C, and the remaining parameters were estimated based on company internal data. The building model as an environment for the RL algorithm was implemented in Python using Gymnasium [39].

Table 1. Model Parameters.

Parameter	Value
C_H	4360 MJ/K
m_{building}	500 t
m_{cheese}	1260 t
c_{building}	480 J/(kg K)
c_{cheese}	3270 J/(kg K)
β	4.938
P_{max}	202,511 W
P_{min}	0 W
T_{ub}	5 °C
T_{lb}	0 °C
Δt	60 s
k_P	500,000 W/K
k_I	2 W/(K s)

2.3. Optimization via Reinforcement Learning

Figure 2 shows the general principle of RL.

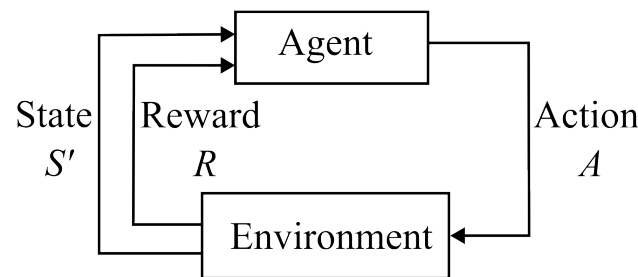


Figure 2. Schematic of agent–environment interface in RL showing the agent, the environment and their interaction via the action, the reward, and the state.

In RL, the problem is framed as a learner (agent) interacting with an environment. The agent interacts with the environment by selecting actions and then receives a reward and a state from the environment. The agent is only aware of the state and a set of possible actions and, based on these, selects an action according to its policy. One iteration of this complete process is called a step, and all steps together are called an episode. The cumulative rewards for the entire episode are called the return. Maximizing the expected return is the goal of the agent. In Q-Learning, the agent estimates the action-value function denoted as $Q(S, A)$. This function gives the expected return when starting in a state S , taking an action A and following a policy π thereafter. The action-value function $Q(S, A)$ is estimated via bootstrapping in Q-learning and is trained via:

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma \max_a Q(S', a) - Q(S, A) \right] \quad (7)$$

where α is the learning rate and γ is a discount factor. In DQL, the action-value function is approximated via a neural network $Q(S, A; \theta)$ with parameter set θ . Known challenges in using function approximation via neural networks in Q-learning are moving targets and maximization bias. These issues are circumvented in DDQL, where two neural networks are used to stabilize the training process. The policy network Q_π is used to select the best action given the current state, while the target network Q_t is used to estimate the value of the following action. The training process is based on mini-batch learning using memory replay [14]. This technique helps to break the correlations of state-action sequences while increasing stability [14]. In contrast to Mnih et al. [14] or Hasselt et al. [16], who use periodic copies of the policy network as the target network or update the networks symmetrically

by switching the roles, respectively, the presented algorithm uses soft updates such as Lillicrap et al. [40]. The applied DDQL algorithm based on [14,16] is shown in Algorithm 2.

Algorithm 2 Double Deep Q-Learning—Training

```

1: Initialize Policy Network  $Q_\pi$  with random weights  $\theta_\pi$ 
2: Initialize Target Network  $Q_t$  with random weights  $\theta_t$ 
3: Initialize Replay Buffer  $M$  as empty buffer
4: loop for each episode:
5:   Reset environment, observe  $S$ 
6:   loop for each step of episode:
7:     With probability  $\epsilon$  select a random action  $A$ 
8:     Otherwise select  $A = \arg \max_a Q_\pi(S, a; \theta_\pi)$ 
9:     Decrease  $\epsilon$ 
10:    Take action  $A$ , observe  $R, S'$ 
11:    Store transition  $(S, A, R, S')$  in  $M$ 
12:    Sample random mini batch  $(s, a, r, s')$  of transitions from  $M$ 
13:    Set  $y = \begin{cases} R, & \text{if } S' \text{ is a terminal state} \\ R + \gamma Q_t(S', \arg \max_{a'} Q_\pi(S', a'; \theta_\pi); \theta_t), & \text{else} \end{cases}$ 
14:    Perform a gradient descent step on the policy network  $\nabla L_\delta(y, Q_\pi(S, A))$ 
15:    Soft update the weights of the target network  $Q_t$ 
16:     $S \leftarrow S'$ 
17:  end loop
18: end loop

```

First, the neural networks are initialized with random starting weights (θ_π and θ_t) and an empty replay buffer M is created. At the start of every episode, the environment needs to be reset. In lines 7–9, exploration (selecting a random action) is performed with a probability of ϵ . Otherwise, exploitation is applied, which means applying the best action according to the current policy (defined by the policy network). To improve the policy π , exploration is essential. As a consequence, the learning process starts with a high exploration rate and later refines the policy using less exploration. The exploration rate is scheduled according to:

$$\epsilon = \epsilon_{\text{end}} + (\epsilon_{\text{start}} - \epsilon_{\text{end}}) e^{-\frac{N_{\text{steps}}}{d_\epsilon}} \quad (8)$$

where ϵ is the current threshold for exploration, ϵ_{start} is the start exploration rate, ϵ_{end} is the end exploration rate, d_ϵ is the decay rate, and N_{steps} is the number of steps done already. In line 10, the action is performed in the environment and the reward R and the next state S' are observed. The transition (S, A, R, S') is stored in the replay buffer and random samples from the replay buffer are used to train the policy network Q_π . As a measure of error in the training of the policy network's weights, the Huber loss L_δ is used:

$$L_\delta(y, f(x)) = \begin{cases} \frac{1}{2}(y - Q_\pi(S, A))^2 & \text{for } |y - Q_\pi(S, A)| \leq \delta \\ \delta \left(|y - Q_\pi(S, A)| - \frac{1}{2}\delta \right), & \text{otherwise} \end{cases} \quad (9)$$

where δ is a parameter of the Huber loss function. The Huber loss acts as the mean squared error for small errors ($y - Q_\pi(S, A)$) and as the mean absolute error for larger errors, and, therefore is more robust to outliers. This loss is used to perform a stochastic gradient descent step using the Adam optimizer to update the weights of the policy network Q_π . The weights of the target network Q_t are updated via soft updates:

$$\theta_t = \tau \theta_\pi + (1 - \tau) \theta_t \quad (10)$$

where τ is the target network's update rate.

This standard RL algorithm can be applied to any problem that can be abstracted in an agent and an environment, where the set of actions is discrete, and the state can be discrete and continuous. The state should fulfill the Markov property, so it should include all information about all aspects of the past agent–environment interaction that make a difference in the future. For physical systems in a linear state space representation, the Markov property is fulfilled.

In the following paragraphs, the application of this algorithm to the DR problem in an industrial warehouse is described. Here, the controller of the refrigeration systems is the agent, and the industrial warehouse is the environment. As an action, the agent can control the state of charge (SOC) of the TES, which is reciprocal to the set temperature of the industrial warehouse. The possible actions are discrete (0–100), where 0 represents the lowest set state of charge and 100 is the highest set state of charge of the TES controlled by the PI controller. The operation is always optimized for one day, so one episode consists of 24 time steps ($\Delta t_{\text{step}} = 1$ h). The state of the environment is given by a set that contains the state of charge, the number of remaining time steps of the day, the electricity price of the next 24 steps and the predicted thermal load $\dot{Q}_{\text{heat},i}$ of the next 24 steps. The electricity price and the thermal load are assumed to be known for one day, that is, for 24 steps. Electricity spot-market prices are usually published one day in advance, so this is a realistic scenario. After the first step, only the 23 next steps are known, and so forth. Unknown values are set to 0. To stabilize the learning process, the variables that define the state are scaled. The temperature is used to calculate the state of charge of the energy storage in %:

$$\text{SOC}_i = 100 \frac{T_{\text{ub}} - T_i}{T_{\text{ub}} - T_{\text{lb}}} \quad (11)$$

The prices p , which are used to calculate the energy costs and the reward, are scaled between one and 10 on a daily basis using:

$$p_i^* = 9 \frac{p_i - \min(p)}{\max(p) - \min(p)} + 1 \quad (12)$$

Scaled values are indicated with *. Prices are scaled due to three reasons: (1) the values of the state should all be in a similar range; (2) electricity prices fluctuate strongly during the year, and the neural network should learn on the relative prices during a day; (3) therefore, the RL agent can handle price signals significantly higher or lower than already seen during training. The scaled prices are always greater than or equal to one, resulting in a positive energy price. If the price could be 0 EUR/(kWh), the RL agent could use the energy during these time slots to increase the energy consumption for free.

The thermal load is scaled to a energy percentage of the energy storage per hour/step via:

$$\dot{Q}_{\text{heat},i}^* = 100 \frac{\dot{Q}_{\text{heat},i} 60 \Delta t}{C_m (T_{\text{ub}} - T_{\text{lb}})} \quad (13)$$

Combining Equations (11)–(13) with $(24 - i)$ to indicate the remaining time steps of a day/episode, this results in a state S_i :

$$S_i = (\text{SOC}_i, (24 - i), p_i^*, \dots, p_{i+23}^*, \dot{Q}_{\text{heat},i}^*, \dots, \dot{Q}_{\text{heat},i+23}^*) \quad (14)$$

Note that if the electricity price and the thermal load for the forecast are not known, a value zero is used. The action space is discrete (0–100) representing the resulting state of charge (SOC_{set}) for the PI controller. The set point temperature can be calculated from the set value of the state of charge by:

$$T_{\text{set}} = T_{\text{lb}} + \frac{(100 - \text{SOC}_{\text{set}})}{100} (T_{\text{ub}} - T_{\text{lb}}) \quad (15)$$

The environment was implemented using Equation (6) in combination with the PI controller. Note that while there are 24 steps for a day, the PI controller logic is executed multiple times (60 times per time step) during a single simulation step of the environment. Then, the average electrical power $P_{el,i}$ during that step is used to calculate the reward R :

$$R_i = -P_{el,i} p_i^* \quad (16)$$

After 24 steps, the terminated flag is set to true and the episode is finished. The Algorithm 2 described was used to train the agent. During training, a random initial state of charge was used in combination with a randomly selected price signal and a randomly selected load signal from the training set. In total, 2000 training episodes were used. During operation, the greedy action (exploitation) is always applied.

The parameters of the RL algorithm can be seen in Table 2.

Table 2. RL Parameters

Parameter	Value
Training episodes	2000
Batch size	1250
Memory buffer size	10,000
Update rate τ	0.005
Adam learning rate	10^{-4}
Initial exploration rate ϵ_{start}	0.9
End exploration rate ϵ_{end}	0.05
Exploration decay rate d_ϵ	2000
Discount factor γ	0.999
Neural net layers	3
Layer 1	(50, 256), ReLu activation
Layer 2	(256, 256), ReLu activation
Layer 3	(256, 101), ReLu activation
Huber loss parameter δ	1

The RL algorithm was implemented in Python and PyTorch [41].

2.4. Validation via Mixed Integer Linear Programming

To validate the performance of RL, MILP was implemented for comparison. Therefore, the same optimization problem as in Section 2.3 was formulated as MILP. MILP is a commonly used optimization algorithm and is ideally suited as a benchmark because the global optimum of an underlying linear problem can be determined. Therefore, the results of the RL-agent can be compared with the optimal solution of the underlying problem. Note that while RL is model-free without prior knowledge of the underlying model, MILP needs complete knowledge of the underlying model. The model was then formulated as MILP as described in the remainder of this subsection. The objective function is minimization of the energy costs and the decision variables are the set point temperatures of the warehouse. The model of the warehouse from Section 2.2 and the PI controller from Algorithm 1 were formulated as constraints. While the warehouse model is linear, the PI controller has a nonlinear effect given by the saturation that limits the controller's output signal. Modeling this effect introduces binary variables to the formulation, and, thereby, drastically increases the computation time. In addition, the PI controller reacts every minute to changes in the warehouse temperature, leading to an optimization problem for one day with 7201 continuous variables and 2880 binary variables. The MILP problem was implemented in Python using Gurobi 11.0 [42]. The complete MILP formulation and its detailed description can be found in Appendix A.

3. Results and Discussion

The trained RL agent was compared with an MILP optimization for the test set, which spans from May 2022 to April 2023. Both, RL and MILP, are used for MPC, always optimizing the operation for a full day at midnight. The currently employed PI controller is simulated and serves as a reference scenario. The results of this case study are shown in Table 3.

Table 3. Optimization results for one year comparing RL and MILP.

Optimization	Costs (EUR)	Savings (EUR)	Relative Savings (%)	Relative Costs (EUR/MWh)
RL	116,831	24,911	17.57	208.00
MILP	115,301	26,441	18.65	205.30
Reference	141,742	-	-	252.10

As expected, MILP has the highest cost reductions, as it leads to the global optimum. While MILP assumes to have perfect knowledge of the complete system dynamics, RL is model-free. In light of this, RL proves to be a promising solution, as it reaches cost reductions of 17.57% compared to 18.65% reached by MILP. These results are especially good considering that RL is model-free, while MILP assumes to have perfect knowledge of the complete model. The detailed optimization for three exemplary days can be seen in Figure 3.

Both, RL and MILP, use low energy prices for the precooling of the warehouse and operate close to the upper boundary T_{ub} of the set temperature. The use of the thermal capacity as energy storage can be seen in the second subplot. The capacity of the industrial warehouse including the stored cheese is so high that the complete temperature band from T_{lb} to T_{ub} is not fully used in these exemplary days. The results of the entire year can be seen in Figure 4.

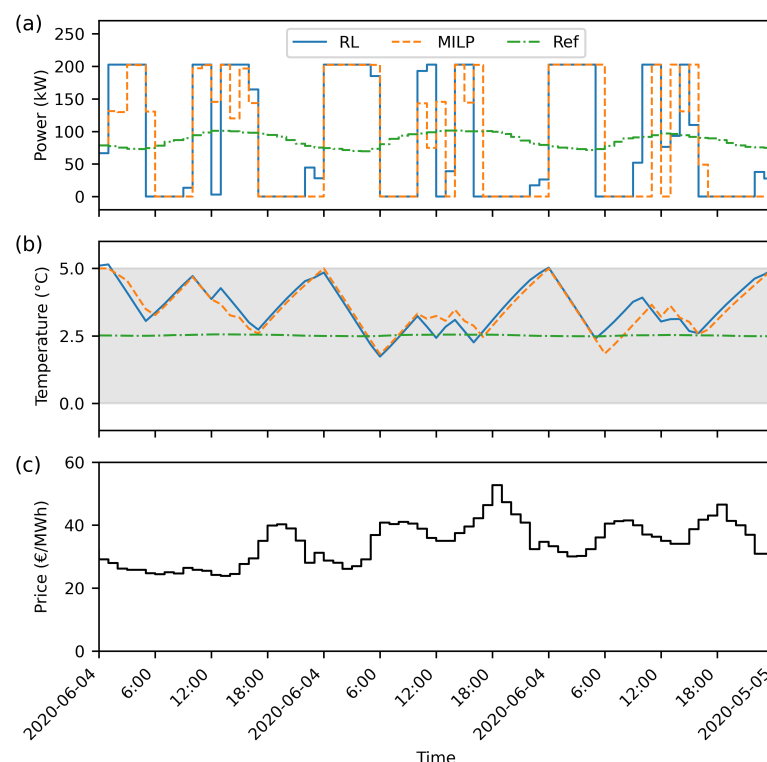


Figure 3. Results for load shifting over three consecutive example days to comparing the RL and MILP with the reference scenario. (a) The refrigeration system's electrical power consumption. (b) Cooling hall temperature variations, where 0 °C corresponds to a fully charged TES and 5 °C represents an empty TES state. (c) The electricity price profile over the same period, illustrating the price-driven adjustments in system operation.

Comparing the weekly savings from Figure 4a with the electricity price from Figure 4c shows that higher prices lead to a greater potential for cost savings. In 2023, where the energy prices were significantly lower than in 2022, the weekly absolute energy savings also were lower. Nevertheless, Figure 4b shows that relative energy savings are high during the entire test period. The only relevant outlier is the Christmas week, where there is no production, a low cooling demand due to cold temperatures, and low energy prices. Therefore, energy costs are low and an average reduction with low energy costs results in high relative (Figure 4b) and low absolute (Figure 4a) energy savings. All in all, Figure 4 shows that both algorithms work for high and low energy prices, as well as for high and low fluctuations. Figure 5 shows the energy price for the test and the training period. There, it can be seen that in the training period the prices are significantly different, in particular lower with less fluctuations, than the prices from the test period. As seen, the presented RL algorithm also works for prices different from those in the data set for training, which can be explained via the scaling in the method. This shows the ability of RL to adapt to new states, encountered by the algorithm before.

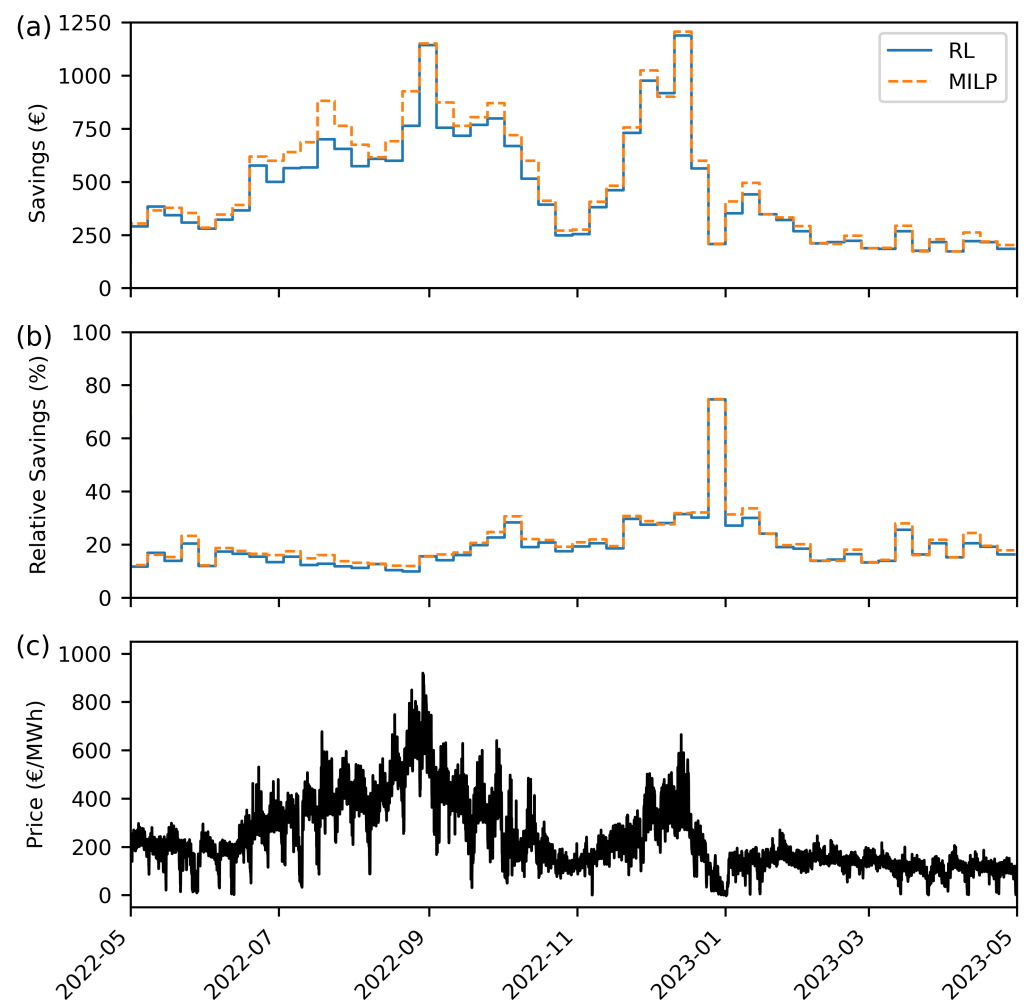


Figure 4. Results for load shifting over the complete year evaluating RL and MILP depending on the electricity price. (a) The weekly energy savings via RL and MILP, (b) the relative weekly energy savings, and (c) the EXAA spot market price for the test period (May 2022 to April 2023).

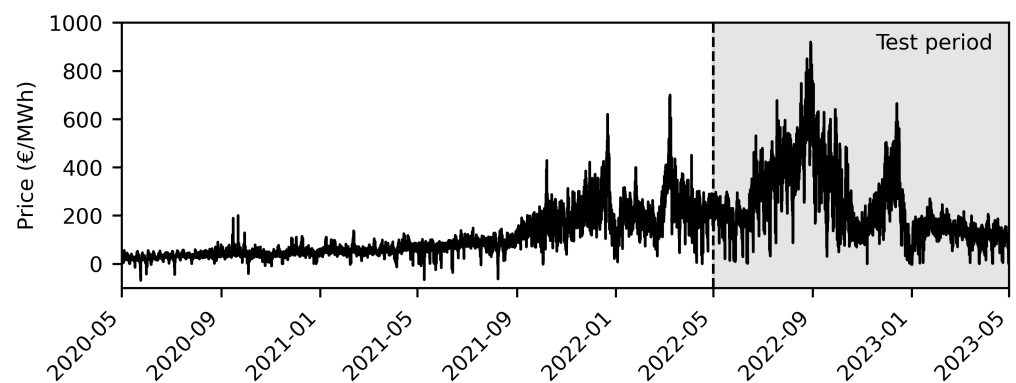


Figure 5. EXAA spot market price from May 2020 to April 2023 showing the electricity prices with its fluctuations. The white area is the training data, whereas the testing period is shaded in grey.

3.1. Evaluation of the RL Training Process

In this subchapter, the RL training process is investigated. The training process for the parameters from Table 2 is shown in Figure 6.

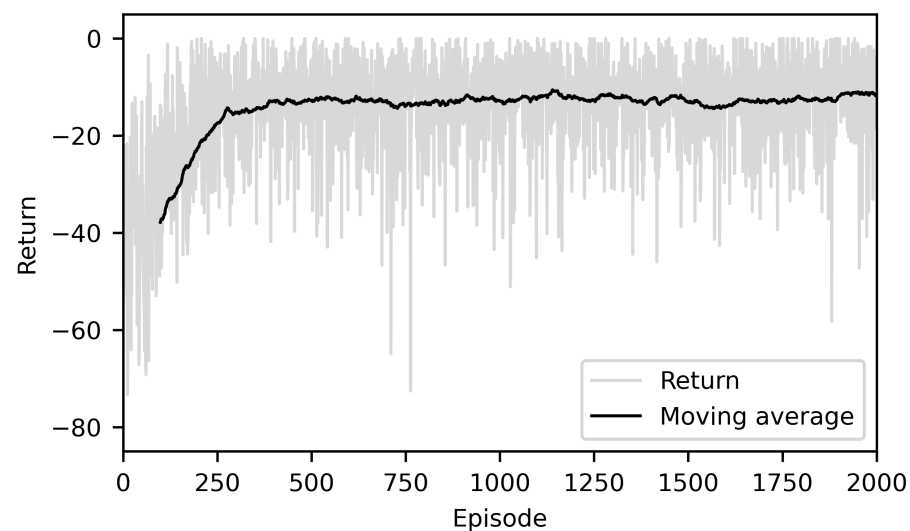


Figure 6. RL training process showing the return of single episodes and the moving average with a window length of 100. The return is proportional to the negative energy costs, while the absolute value is irrelevant and only chosen to be an appropriate scale for the training process. Maximizing the negative energy costs is equal to minimizing the energy costs.

The RL agent is improving rapidly with decent results after 250 training episodes. The high fluctuation of the results of single episodes is caused by the random initialization of the state of charge. In some episodes, the energy storage starts full and ends empty, so no additional energy is needed, resulting in a reward of EUR 0. Still, for training different start values are beneficial. To further investigate the training process, different values for the training episodes and the exploration decay rate d_ϵ are shown in Figure 7. The number of training episodes and d_ϵ were tested with values from 100 to 10,000.

Figure 7 shows that the training process converges and, after 250 episodes, good results can be achieved. However, using longer training periods on average improves the results even further. While 250 episodes average 16.94%, 10,000 training episodes average a cost reduction of 17.87% and the deviation of the results is much smaller.

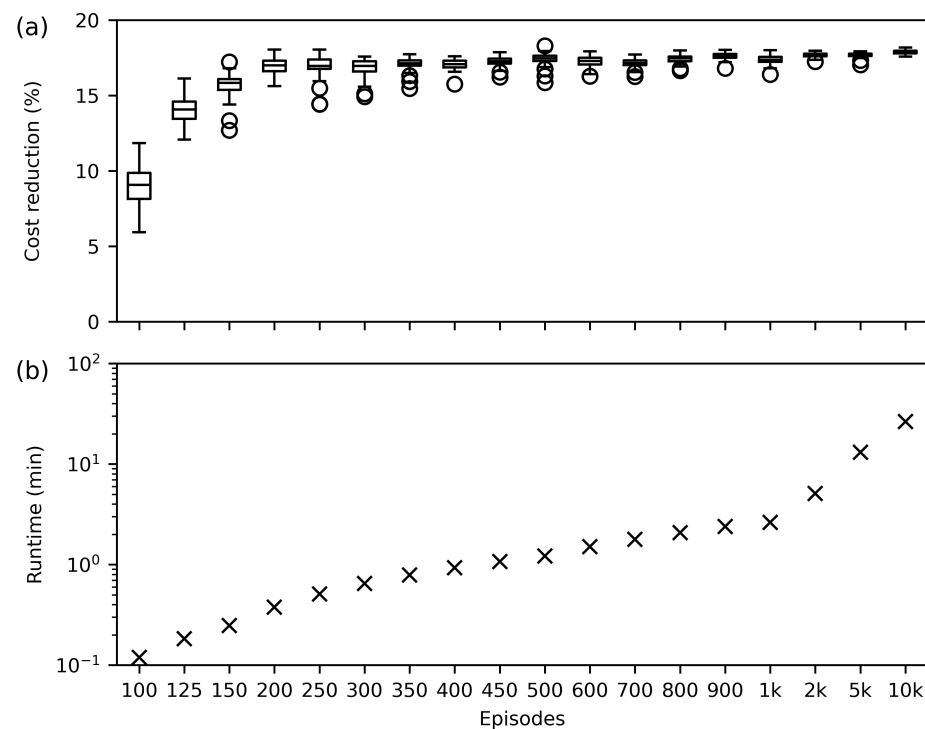


Figure 7. RL training process showing the cost reduction (a) and runtime (b) of different training period lengths.

3.2. Analysis of Computational Complexity

MILP is capable of finding cost-optimal solutions; however, it can be computationally intensive due to the large number of variables involved. The original problem formulation contains 7201 continuous variables and 2880 binary variables. After applying Gurobi's presolve, the number of continuous variables is reduced to 5070, while the number of binary variables remains unchanged. Despite this reduction, the average computation time per optimization step is 19 s, which is reasonable for complex control tasks. In contrast, the RL agent operates significantly faster, with an average computation time of 2 ms per optimization step (excluding the training phase). Training the RL model requires 306 s (5 min) for 2000 episodes. As a result, the total runtime, including both training and operation, is 307 s, which is substantially less than the total runtime for MILP, which is 6963 s (116 min). Although MILP provides competitive results with a manageable average optimization time of 19 s, the RL approach offers significantly faster performance, being approximately 23 times faster than MILP. Note that all runtime tests were conducted on a 2022 MacBook Pro M2.

3.3. Practical Applicability and Future Research Directions

The proposed RL approach is designed to be versatile, making it suitable for a wide range of DR applications, managing energy-storage systems ranging from batteries to TES systems such as boilers, chilled water buffers, or hot water heat pumps. These energy-storage systems can be described in state space via the energy stored, similar to the thermal mass of the food-production plant. Therefore, the same state formulation as in Equation (14) can be applied to train the RL agent. Although the state of charge can be directly used in the case of batteries, in thermal applications, the state is calculated using the temperature as in Equation (11). Then, the proposed RL algorithm can be applied to reduce the energy costs in any load-shifting use case with an energy storage in an RTP-driven scenario, where a forecast of the load profile is known and historical data are available to train the RL agent. If a system is dependent on additional inputs, such as the ambient temperature, these inputs can be added to the state (in a similar way as the price and the thermal load are considered in the

study presented). To optimize with respect to energy usage, the price would be neglected (i.e., excluded from the state) and the energy consumption would be defined as the reward. This shows the broad applicability of the method to a wide range of DR applications.

In practical industrial applications, due to missing sensors and missing connections between energy-management systems and the production-management systems [13], data collection is often a challenge. Furthermore, data uncertainty poses one of the main challenges for industrial DR identified by Zhang and Grossmann [13], which has been addressed in the literature via stochastic optimization [43]. Data collection in the cloud or compliance with the ISO 50001 standard [44] can help solve this challenge. However, collecting data for multiple months, at least hundreds of episodes, each representing a full day, can be inefficient and impractical. To help address this issue, transfer learning can be employed. Through this method, the agent is pre-trained using simulated environments and later deployed to real world applications. This not only speeds up the learning process, but also improves safety and system stability by allowing early training phases to exclude extreme actions and critical states. However, transfer learning requires careful analysis of the model used as an environment to ensure that it is capable of representing the real world application. Otherwise, misleading policies may be learned.

In addition to a solid data base, the optimization method also depends on load forecasts. A perfect prediction, as assumed in the current study, neglects the uncertainty in load forecasting. This could be addressed by applying ML-based algorithms available for load forecasting such as Long Short-Term Memory (LSTM) [45], Transformer [46] or Extended Long Short-Term Memory (xLSTM) [47].

Another key aspect of the RL framework is its integration with existing control systems. Instead of directly controlling the cooling power, the RL algorithm adjusts the set point temperature of an existing PI controller. This indirect control improves the overall system stability and makes the solution easier to implement. Directly controlling the chiller power is often impractical, making our approach more feasible for real-world applications. Also, using the environment set up to train the agent, the control parameters of the existing system could be further improved [48], e.g., by dynamical parameter adjustments.

The RL algorithm's computation is efficient, requiring only 2 ms per optimization. This makes it ideal for edge computing environments, such as Internet of Things (IoT) devices, where fast decision-making is essential. Its compatibility with microcontrollers that support Python, combined with the use of open-source libraries like PyTorch, further reduces implementation costs by eliminating the need for expensive proprietary software licenses. In contrast, traditional approaches such as MILP would require high-performance computing resources and expensive commercial solvers, such as Gurobi, making RL a more accessible and cost-effective alternative for DR applications.

4. Conclusions

Deep reinforcement learning was employed as a data-driven method for energy load shifting in a food-processing plant. The food-processing plant was modeled based on a transient energy balance and the thermal capacities of the food stored and the building were used as flexibility for load shifting of the industrial refrigeration system. The load is shifted via optimizing a PI controller's set point temperature and, thereby, safe states of the system are ensured. To optimize load shifting, an RL algorithm was compared with a state-of-the-art MPC controller based on MILP. As an RL algorithm, DDQL was applied. The results of the simulation study show cost savings in an RTP-driven scenario of 17.57% via model-free RL, compared to 18.65% via MILP, which assumes perfect knowledge of the model. This demonstrates that RL is capable of providing good solutions without prior knowledge of the model. Also, RL proves to be computationally highly efficient. Even when accounting for both training and operation, the RL agent is still 23 times faster than the MILP method used for comparison. Training the agent only needs to be done once, its operation being fast, with an average computation time of only 2 ms, compared to 19 s for solving the MILP. Additionally, the RL agent is based on open source code, which does not require expensive commercial

licenses. The computational efficiency and independence of expensive licensing make RL a promising tool to optimize energy-management problems, especially for edge computing on IoT devices. Due to the adaptive and self-learning capabilities of RL, this method can also be transferred to various energy-management applications.

Author Contributions: Conceptualization, P.W., S.H., E.E., M.K. and P.K.; methodology, P.W., S.H., E.E. and P.K.; software, P.W.; validation, P.W., P.K. and S.H.; formal analysis, P.W., S.H., E.E. and P.K.; investigation, P.W.; resources, P.K.; data curation, P.W.; writing—original draft preparation, P.W.; writing—review and editing, P.W., S.H., E.E., M.K. and P.K.; visualization, P.W. and E.E.; supervision, M.K. and P.K.; project administration, P.K.; funding acquisition, P.K. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Austrian Federal Ministry for Digital and Economic Affairs, the National Foundation for Research, Technology, and Development as well as the Christian Doppler Research Association.

Data Availability Statement: The datasets presented in this article are not readily available because they include confidential data from our project partner Rupp Austria GmbH. Requests to access the datasets should be directed to peter.kepplinger@fhv.at.

Acknowledgments: The authors are grateful to the project partner Rupp Austria GmbH for providing the data and all the fruitful discussions.

Conflicts of Interest: Sebastian Hegenbart is from Digital Factory Vorarlberg GmbH. The other authors declare no conflict of interest.

Abbreviations

The following abbreviations are used in this manuscript:

DR	Demand response
DDQL	Double deep Q-learning
DDPG	Deep deterministic policy gradient
DQL	Deep Q-learning
DQN	Deep Q-networks
DRL	Deep reinforcement learning
DSM	Demand side management
EXAA	Energy exchange Austria
HVAC	Heating, ventilation and air conditioning
IoT	Internet of things
LP	Linear programming
LSTM	Long Short-Term Memory
MILP	Mixed integer linear programming
MINLP	Mixed integer nonlinear programming
ML	Machine learning
MPC	Model predictive control
PI	Proportional integral
PPO	Proximal policy optimization
PV	Photovoltaic
RL	Reinforcement learning
RTP	Real-time pricing
SAC	Soft actor-critic
TES	Thermal energy storage
TOU	Time-of-use
xLSTM	Extended Long Short-Term Memory

Appendix A. MILP Formulation

The following decision variables are defined for the model:

- $T_{\text{set},p}$ is the set point temperature of the warehouse during the time period p .
- $T_{H,t}$ is the warehouse temperature at the time point t .

- u_p is the output signal of the P-controller before saturation during the time period p .
- $u_{tmp,p}$ is a helper variable for calculating the saturation during the time period p .
- $b_{1,p}$ $b_{2,p}$ are binary variables to calculate the saturation during the time period p .
- $P_{el,p}$ is the electrical power consumption of the industrial refrigeration system p .

Note that $T_{set,p}$ is the actual decision variable that is used and the remaining variables are helper variables to improve the readability of the formulation. Also, $T_{set,p}$ is a continuous variable in the MILP formulation to improve the computation time. Therefore, the MILP problem has slightly more flexibility available for load shifting than the RL problem, where $T_{set,p}$ is an integer. The following time series are needed as inputs:

- π_p is the price signal during the time period p .
- $\dot{Q}_{heat,p}$ is the heat flow rate of the load during the time period p .

The following parameters are needed additionally:

- Δt is the length of a time period.
- $T_{H,start}$ is the initial warehouse temperature at the time point 0.
- β is the energy efficiency ratio of the industrial refrigeration system.
- k_P proportional factor of the controller.
- k_I integral factor of the controller.
- P_{min} is the minimum electrical power.
- P_{max} is the maximum electrical power.
- T_{lb} is the minimum set point temperature.
- T_{ub} is the maximum set point temperature.
- N is the number of time periods.
- M_1 and M_2 are big M constraints.

The following sets are defined:

- $\mathcal{P}$ is the set of time period indices.
- $\mathcal{T}$ is the set of time point indices.
- $\mathcal{I}$ is a set to index of every hour of a day.
- $\mathcal{J}$ is a set to index every minute in a hour.

The optimization problem minimizing the total costs can be written as:

$$\text{obj: } \min_{T_{set}} \sum_{p=0}^{N-1} P_{el,p} \pi_p \Delta t \quad (\text{A1})$$

subject to:

$$\mathcal{P} = \{0, \dots, N-1\} \quad (\text{A2})$$

$$\mathcal{T} = \{0, \dots, N\} \quad (\text{A3})$$

$$T_{H,0} = T_{H,start} \quad (\text{A4})$$

$$\forall t \in \mathcal{T} \setminus 0: T_{H,t} = T_{H,t-1} + \frac{1}{C_H} (\dot{Q}_{heat,t-1} - P_{el,t-1} \beta) \Delta t \quad (\text{A5})$$

$$\forall t \in \mathcal{T}: T_{H,t} \leq T_{ub} \quad (\text{A6})$$

$$\forall t \in \mathcal{T}: T_{H,t} \geq T_{lb} \quad (\text{A7})$$

$$u_0 = k_0 (T_{H,0} - T_{set,0}) \quad (\text{A8})$$

$$\forall p \in \mathcal{P} \setminus 0: u_p = k_P (T_{H,p} - T_{set,p}) + (k_I \Delta t - k_P) (T_{H,p-1} - T_{set,p-1}) + u_{p-1} \quad (\text{A9})$$

$$\forall p \in \mathcal{P}: u_{tmp,p} \leq u_p \quad (\text{A10})$$

$$\forall p \in \mathcal{P}: u_{tmp,p} \leq P_{max} \quad (\text{A11})$$

$$\forall p \in \mathcal{P}: u_{tmp,p} \geq u_p - M_1 b_{1,p} \quad (\text{A12})$$

$$\forall p \in \mathcal{P}: u_{tmp,p} \geq P_{max} - M_1 (1 - b_{1,p}) \quad (\text{A13})$$

$$\forall p \in \mathcal{P} : P_{el,p} \geq u_{tmp,p} \quad (A14)$$

$$\forall p \in \mathcal{P} : P_{el,p} \geq P_{\min} \quad (A15)$$

$$\forall p \in \mathcal{P} : P_{el,p} \leq u_{tmp,p} + M_2 b_{2,p} \quad (A16)$$

$$\forall p \in \mathcal{P} : P_{el,p} \leq P_{\min} + M_2(1 - b_{2,p}) \quad (A17)$$

$$\forall p \in \mathcal{P} : T_{set,p} \geq T_{lb} \quad (A18)$$

$$\forall p \in \mathcal{P} : T_{set,p} \leq T_{ub} \quad (A19)$$

$$\mathcal{I} = \{0, \dots, 23\} \quad (A20)$$

$$\mathcal{J} = \{0, \dots, 59\} \quad (A21)$$

$$\forall i \in \mathcal{I}, \forall j \in \mathcal{J} : T_{set,60i} = T_{set,60i+j} \quad (A22)$$

$$\forall p \in \mathcal{P}, \forall t \in \mathcal{T} : T_{set,p}, T_{H,t}, u_p, u_{tmp,p}, P_{el,p} \in \mathbb{R} \quad (A23)$$

$$\forall p \in \mathcal{P} : b_{1,p}, b_{2,p} \in \{0, 1\} \quad (A24)$$

where Equation (A1) is the objective function minimizing the total energy costs. The variables are split into state variables (at a certain time point t) and input variables (during a certain time period p). Equation (A2) defines a set of indices for every time period and Equation (A3) defines a set of indices for every time point. The start temperature of the warehouse is defined in Equation (A4) and the remaining temperatures are calculated via an energy balance in Equation (A5). Equations (A6) and (A7) introduce additional cutting planes, which are not considered in the RL scenario, designed to enhance computational efficiency and accelerate the solution process. The remaining Equations (A8)–(A24) describe the PI controller with saturation. Equation (A8) is the initial condition of the PI controller assuming that that integral error of the historic values is zero, and, therefore, only the proportional factor k_p is needed for the first controller output u_0 . Equation (A9) is used to calculate the controller output signal u_p of the remaining time steps depending on the proportional factor k_p and the integral factor k_I . Equations (A10)–(A17) represent the saturation element. Saturation is used to limit the output signal of the controller between $P_{\min}$ and $P_{\max}$. For implementation, the big M method is used. The set point temperature $T_{set,p}$ is limited in Equations (A18) and (A19). The PI controller calculates a new output signal u_p at every time point. To reduce high-frequency output signals, the $T_{set,p}$ can only be changed every hour. So Equations (A20)–(A22) are used to ensure that during every hour, all set points are the same. This could also be done by defining a separate set of indices with a different time period, which would decrease the number of variables. But for readability reasons and because simple variable reductions are in any case carried out by the solver, this was not applied. The final optimization problem consists of $5N + 1$ continuous and $2N$ binary variables. For one simulated day with $N = 1440$ these are 7201 continuous and 2880 binary variables.

References

1. Clairand, J.-M.; Briceno-Leon, M.; Escrive-Escrive, G.; Pantaleo, A. M. Review of Energy Efficiency Technologies in the Food Industry: Trends, Barriers, and Opportunities. *IEEE Access* **2020**, *8*, 48015–48029. [\[CrossRef\]](#)
2. Panda, S.; Mohanty, S.; Rout, P.K.; Sahu, B.K.; Parida, S.; Samanta, I.S.; Bajaj, M.; Piecha, M.; Blazek, V.; Prokop, L. A comprehensive review on demand side management and market design for renewable energy support and integration. *Energy Rep.* **2023**, *10*, 2228–2250. [\[CrossRef\]](#)
3. Siddiquee, S.M.S.; Howard, B.; Bruton, K.; Brem, A.; O’Sullivan, D.T.J. Progress in Demand Response and It’s Industrial Applications. *Front. Energy Res.* **2021**, *9*, 673176. [\[CrossRef\]](#)
4. Morais, D.; Gaspar, P.D.; Silva, P.D.; Andrade, L.P.; Nunes, J. Energy Consumption and Efficiency Measures in the Portuguese Food Processing Industry. *J. Food Process. Preserv.* **2022**, *46*, e14862. [\[CrossRef\]](#)
5. Koohi-Fayegh, S.; Rosen, M.A. A Review of Energy Storage Types, Applications and Recent Developments. *J. Energy Storage* **2020**, *27*, 101047. [\[CrossRef\]](#)
6. Chen, C.; Sun, H.; Shen, X.; Guo, Y.; Guo, Q.; Xia, T. Two-Stage Robust Planning-Operation Co-Optimization of Energy Hub Considering Precise Energy Storage Economic Model. *Appl. Energy* **2019**, *252*, 1. [\[CrossRef\]](#)

7. Giordano, L.; Furlan, G.; Puglisi, G.; Cancellara, F.A. Optimal Design of a Renewable Energy-Driven Polygeneration System: An Application in the Dairy Industry. *J. Clean. Prod.* **2023**, *405*, 136933. [\[CrossRef\]](#)
8. Pazmiño-Arias, A.; Briceño-León, M.; Clairand, J.-M.; Serrano-Guerrero, X.; Escrivá-Escrivá, G. Optimal Scheduling of a Dairy Industry Based on Energy Hub Considering Renewable Energy and Ice Storage. *J. Clean. Prod.* **2023**, *429*, 139580. [\[CrossRef\]](#)
9. Cirocco, L.; Pudney, P.; Riahi, S.; Liddle, R.; Semsarilar, H.; Hudson, J.; Bruno, F. Thermal Energy Storage for Industrial Thermal Loads and Electricity Demand Side Management. *Energy Convers. Manag.* **2022**, *270*, 116190. [\[CrossRef\]](#)
10. Saffari, M.; de Gracia, A.; Fernández, C.; Belusko, M.; Boer, D.; Cabeza, L.F. Optimized Demand Side Management (DSM) of Peak Electricity Demand by Coupling Low Temperature Thermal Energy Storage (TES) and Solar PV. *Appl. Energy* **2018**, *211*, 604–616. [\[CrossRef\]](#)
11. Wohlgenannt, P.; Huber, G.; Rheinberger, K.; Preißinger, M.; Kepplinger, P. Modelling of a Food-Processing Plant for Industrial Demand Side Management. In Proceedings of the HEAT POWERED CYCLES 2021 Conference Proceedings, Bilbao, Spain, 10–13 April 2022; pp. 638–649. [\[CrossRef\]](#)
12. Wohlgenannt, P.; Huber, G.; Rheinberger, K.; Kolhe, M.; Kepplinger, P. Comparison of Demand Response Strategies Using Active and Passive Thermal Energy Storage in a Food-Processing Plant. *Energy Rep.* **2024**, *12*, 226–236. [\[CrossRef\]](#)
13. Zhang, Q.; Grossmann, I.E. Enterprise-Wide Optimization for Industrial Demand Side Management: Fundamentals, Advances, and Perspectives. *Chem. Eng. Res. Des.* **2016**, *116*, 114–131. [\[CrossRef\]](#)
14. Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A.A.; Veness, J.; Bellemare, M.G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; et al. Human-Level Control through Deep Reinforcement Learning. *Nature* **2015**, *518*, 529–533. [\[CrossRef\]](#) [\[PubMed\]](#)
15. Watkins, C.J.C.H.; Dayan, P. Q-Learning. *Mach. Learn.* **1992**, *8*, 279–292. [\[CrossRef\]](#)
16. van Hasselt, H.; Guez, A.; Silver, D. Deep Reinforcement Learning with Double Q-learning. In Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence (AAAI-16), Phoenix, AZ, USA, 12–17 February 2016; pp. 2094–2100. [\[CrossRef\]](#)
17. Vázquez-Canteli, J.R.; Nagy, Z. Reinforcement Learning for Demand Response: A Review of Algorithms and Modeling Techniques. *Appl. Energy* **2019**, *235*, 1072–1089. [\[CrossRef\]](#)
18. Yu, L.; Qin, S.; Zhang, M.; Shen, C.; Jiang, T.; Guan, X. A Review of Deep Reinforcement Learning for Smart Building Energy Management. *IEEE Internet Things J.* **2021**, *8*, 12046–12063. [\[CrossRef\]](#)
19. Lazic, N.; Boutilier, C.; Lu, T.; Wong, E.; Roy, B.; Ryu, M.; Imwalle, G. Data Center Cooling Using Model-Predictive Control. In *Advances in Neural Information Processing Systems*; Curran Associates, Inc.: New York, NY, USA, 2018; Volume 31. Available online: https://proceedings.neurips.cc/paper_files/paper/2018/file/059fdcd96baeb75112f09fa1dcc740cc-Paper.pdf (accessed on 23 September 2024).
20. Part 2: Kinds of RL Algorithms—Spinning Up Documentation. Available online: https://spinningup.openai.com/en/latest/spinningup/rl_intro2.html (accessed on 23 September 2024).
21. Afroosheh, S.; Esapour, K.; Khorram-Nia, R.; Karimi, M. Reinforcement Learning Layout-Based Optimal Energy Management in Smart Home: AI-Based Approach. *IET Gener. Transm. Distrib.* **2024**, *18*, 2509–2520. [\[CrossRef\]](#)
22. Lissa, P.; Deane, C.; Schukat, M.; Seri, F.; Keane, M.; Barrett, E. Deep Reinforcement Learning for Home Energy Management System Control. *Energy AI* **2021**, *3*, 100043. [\[CrossRef\]](#)
23. Liu, Y.; Zhang, D.; Gooi, H.B. Optimization Strategy Based on Deep Reinforcement Learning for Home Energy Management. *CSEE J. Power Energy Syst.* **2020**, *6*, 572–582. [\[CrossRef\]](#)
24. Jiang, Z.; Risbeck, M. J.; Ramamurti, V.; Murugesan, S.; Amores, J.; Zhang, C.; Lee, Y.M.; Drees, K.H. Building HVAC Control with Reinforcement Learning for Reduction of Energy Cost and Demand Charge. *Energy Build.* **2021**, *239*, 110833. [\[CrossRef\]](#)
25. Brandi, S.; Piscitelli, M.S.; Martellacci, M.; Capozzoli, A. Deep Reinforcement Learning to Optimise Indoor Temperature Control and Heating Energy Consumption in Buildings. *Energy Build.* **2020**, *224*, 110225. [\[CrossRef\]](#)
26. Brandi, S.; Fiorentini, M.; Capozzoli, A. Comparison of Online and Offline Deep Reinforcement Learning with Model Predictive Control for Thermal Energy Management. *Autom. Constr.* **2022**, *135*, 104128. [\[CrossRef\]](#)
27. Coraci, D.; Brandi, S.; Capozzoli, A. Effective Pre-Training of a Deep Reinforcement Learning Agent by Means of Long Short-Term Memory Models for Thermal Energy Management in Buildings. *ENergy Convers. Manag.* **2023**, *291*, 117303. [\[CrossRef\]](#)
28. Han, G.; Lee, S.; Lee, J.; Lee, K.; Bae, J. Deep-Learning- and Reinforcement-Learning-Based Profitable Strategy of a Grid-Level Energy Storage System for the Smart Grid. *J. Energy Storage* **2021**, *41*, 102868. [\[CrossRef\]](#)
29. Muriithi, G.; Chowdhury, S. Deep Q-Network Application for Optimal Energy Management in a Grid-Tied Solar PV-Battery Microgrid. *J. Eng.* **2022**, *2022*, 422–441. [\[CrossRef\]](#)
30. Lu, R.; Hong, S.H. Incentive-Based Demand Response for Smart Grid with Reinforcement Learning and Deep Neural Network. *Appl. Energy* **2019**, *236*, 937–949. [\[CrossRef\]](#)
31. Brandi, S.; Coraci, D.; Borello, D.; Capozzoli, A. Energy Management of a Residential Heating System Through Deep Reinforcement Learning. In *Sustainability in Energy and Buildings 2021; Smart Innovation, Systems and Technologies*; Littlewood, J.R., Howlett, R.J., Jain, L.C., Eds.; Springer Nature Singapore: Singapore, 2022; Volume 263, pp. 329–339. [\[CrossRef\]](#)
32. Brandi, S.; Gallo, A.; Capozzoli, A. A Predictive and Adaptive Control Strategy to Optimize the Management of Integrated Energy Systems in Buildings. *Energy Rep.* **2022**, *8*, 1550–1567. [\[CrossRef\]](#)
33. Silvestri, A.; Coraci, D.; Brandi, S.; Capozzoli, A.; Borkowski, E.; Köhler, J.; Wu, D.; Zeilinger, M.N.; Schlueter, A. Real Building Implementation of a Deep Reinforcement Learning Controller to Enhance Energy Efficiency and Indoor Temperature Control. *Appl. Energy* **2024**, *368*, 123447. [\[CrossRef\]](#)

34. Gao, G.; Li, J.; Wen, Y. DeepComfort: Energy-Efficient Thermal Comfort Control in Buildings Via Reinforcement Learning. *IEEE Internet Things J.* **2020**, *7*, 8472–8484. [\[CrossRef\]](#)
35. Opalic, S.M.; Palumbo, F.; Goodwin, M.; Jiao, L.; Nielsen, H.K.; Kolhe, M.L. COST-WINNERS: COST Reduction with Neural NETworks-Based Augmented Random Search for Simultaneous Thermal and Electrical Energy Storage Control. *J. Energy Storage* **2023**, *72*, 108202. [\[CrossRef\]](#)
36. Azuatalam, D.; Lee, W.-L.; de Nijs, F.; Liebman, A. Reinforcement Learning for Whole-Building HVAC Control and Demand Response. *Energy AI* **2020**, *2*, 100020. [\[CrossRef\]](#)
37. Li, Z.; Sun, Z.; Meng, Q.; Wang, Y.; Li, Y. Reinforcement Learning of Room Temperature Set-Point of Thermal Storage Air-Conditioning System with Demand Response. *Energy Build.* **2022**, *259*, 111903. [\[CrossRef\]](#)
38. DAY-AHEAD PREISE. Available online: <https://markttransparenz.apg.at/de/markt/Markttransparenz/Uebertragung/EXAA-Spotmarkt> (accessed on 23 September 2024).
39. Gymnasium Version 0.29.1. Available online: <https://pypi.org/project/gymnasium/> (accessed on 23 September 2024).
40. Lillicrap, T.P.; Hunt, J.J.; Pritzel, A.; Heess, N.; Erez, T.; Tassa, Y.; Silver, D.; Wierstra, D. Continuous Control with Deep Reinforcement Learning. *arXiv* **2019**, arXiv:1509.02971.
41. Pytorch Version 2.1.1. Available online: <https://pytorch.org> (accessed on 23 September 2024).
42. Gurobi Version 11.0. Available online: <https://www.gurobi.com> (accessed on 23 September 2024).
43. Zhang, H.; Li, Z.; Xue, Y.; Chang, X.; Su, J.; Wang, P.; Guo, Q.; Sun, H. A Stochastic Bi-Level Optimal Allocation Approach of Intelligent Buildings Considering Energy Storage Sharing Services. *IEEE Trans. Consum. Electron.* **2024**, *70*, 5142–5153. [\[CrossRef\]](#)
44. ISO Standard No. 50001; Energy Management. International Organization for Standardization: London, UK, 2018.
45. Hochreiter, S.; Schmidhuber, J. Long Short-Term Memory. *Neural Comput.* **1997**, *9*, 1735–1780. [\[CrossRef\]](#)
46. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention is All You Need. In Proceedings of the 31st International Conference on Neural Information Processing Systems—NIPS’17, Long Beach, CA, USA, 4–9 December 2017; Curran Associates Inc.: Red Hook, NY, USA, 2017; pp. 6000–6010.
47. Beck, M.; Pöppel, K.; Spanring, M.; Auer, A.; Prudnikova, O.; Kopp, M.; Klambauer, G.; Brandstetter, J.; Hochreiter, S. xLSTM: Extended Long Short-Term Memory. *arXiv* **2024**, arXiv:2405.04517.
48. Zhang, H.; Zhai, X.; Zhang, J.; Bai, X.; Li, Z. Mechanism Analysis of the Effect of the Equivalent Proportional Coefficient of Inertia Control for a Doubly Fed Wind Generator on Frequency Stability in Extreme Environments. *Sustainability* **2024**, *16*, 4965. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.